



Resumo Analítico – IPW (LEIC32D)

Introdução à Programação Web · 2024/2025 · ISEL

Repositório: github.com/isel-leic-ipw/2425i-IPW-LEIC32D

Este documento consolida os apontamentos das 14 aulas (lições) da unidade curricular, respeitando a ordem, conceitos, exemplos e ligações entre os temas. Inclui ainda uma lista de termos-chave para referência rápida.

Lesson 01 – Introdução à Web e HTTP

Tema e Objetivos

Estabelecer os fundamentos da World Wide Web e do protocolo HTTP, essenciais para compreender a comunicação cliente-servidor.

Conceitos e Definições

- **World Wide Web (WWW):** Sistema de informação onde recursos são identificados por URLs, interligados por hiperligações.
- **URL:** Endereço único. Estrutura: `protocolo://domínio:porta/caminho?query#fragmento`.
- **HTTP (Hypertext Transfer Protocol):** Protocolo de camada de aplicação, modelo *pedido-resposta*.
- **Métodos HTTP:** GET , POST , PUT , DELETE , PATCH , HEAD , OPTIONS .
- **Códigos de estado:**
 - 1xx – Informativo
 - 2xx – Sucesso (200 OK, 201 Created)
 - 3xx – Redirecionamento
 - 4xx – Erro do cliente (400, 404)
 - 5xx – Erro do servidor (500)
- **Cabeçalhos HTTP:** Metadados (Content-Type , User-Agent , Set-Cookie).

Exemplos e Dados

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
```

Referências: RFC 7230-7235, IANA registros.

Ligações

Base para **Express.js** (Lesson 08) e APIs REST (Lesson 12).

Lesson 02 – HTML Fundamentos

Conceitos

- **HTML:** Linguagem de marcação baseada em *tags*.
- **Elemento:** tag de abertura, conteúdo, fecho. Elementos vazios: `<img>` .
- **Atributos:** `href` , `src` , `alt` , `class` , `id` .
- **Estrutura HTML5:** `<!DOCTYPE html>` , `<html>` , `<head>` , `<body>` .
- **Elementos semânticos:** `<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<footer>` .

Exemplo prático

```
<header>
  <h1>O Meu Website</h1>
  <nav>
    <ul>
      <li><a href="/">Home</a></li>
    </ul>
  </nav>
</header>
<main>
  <section>
    <h2>Sobre Mim</h2>
    <p>Estudante de LEIC no ISEL.</p>
  </section>
</main>
<footer>© 2024</footer>
```

Ligações

O HTML é a base para o **CSS** (Lesson 04) e manipulação do **DOM** (Lesson 05).

Lesson 03 – HTML Avançado e Formulários

Conceitos

- **<form>** : atributos `action` , `method` , `enctype` .
- **Campos <input>** : tipos `text` , `password` , `email` , `number` , `date` , `checkbox` , `radio` , `file` .
- **Outros:** `<label>` , `<textarea>` , `<select>` , `<button>` , `<fieldset>` .
- **Validação nativa:** atributos `required` , `minlength` , `pattern` .

Exemplo de formulário

```
<form action="/submit" method="post">
  <label for="nome">Nome:</label>
  <input type="text" id="nome" name="nome" required minlength="3">

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <button type="submit">Enviar</button>
</form>
```

Referências

MIME types: application/x-www-form-urlencoded , multipart/form-data .

Lesson 04 – CSS Fundamentos

Conceitos

- **Regra CSS:** seletor { propriedade: valor; }
- **Inclusão:** inline, interna (<style>), externa (<link>).
- **Seletores:** elemento, classe (.), ID (#), pseudo-classes (:hover), combinadores.
- **Box Model:** content → padding → border → margin .
- **Especificidade:** inline (1000) > ID (100) > classe (10) > elemento (1).
- **Unidades:** px , em , rem , vw , vh .

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
}
.destaque {
  background-color: yellow;
  font-weight: bold;
}
#menu-principal a:hover {
  color: red;
}
```

Lesson 04-extra – CSS Layout: Flexbox e Grid

Flexbox

- **Contentor:** `display: flex;`
- **Propriedades:** `flex-direction` , `justify-content` , `align-items` , `flex-wrap` .
- **Itens:** `flex-grow` , `flex-shrink` , `flex-basis` .

Grid

- **Contentor:** `display: grid;`
- **Definição:** `grid-template-columns: 1fr 2fr;` `grid-template-areas` .
- **Funções:** `repeat()` , `minmax()` .

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
}
```

Lesson 05 – JavaScript: Sintaxe e DOM

Conceitos

- **Variáveis:** `let` , `const` (evitar `var`).
- **Tipos:** `number` , `string` , `boolean` , `null` , `undefined` , `object` , `array` .
- **Funções:** declaração , expressão , arrow functions `() => {}` .
- **DOM:** `document.querySelector()` , `getElementById()` .
- **Manipulação:** `textContent` , `innerHTML` , `classList` , `createElement` , `appendChild` .

```
const btn = document.getElementById('add-btn');
btn.addEventListener('click', () => {
  const novoItem = document.createElement('li');
  novoItem.textContent = 'Novo item';
  lista.appendChild(novoItem);
});
```

Lesson 06 – JavaScript: Eventos e Assincronia

Eventos

- `addEventListener`, objeto `event`, `preventDefault()`, `stopPropagation()`.
- **Delegação de eventos:** aproveitar o *bubbling*.

Assincronia

- **Callbacks** → **Promises** (`.then/.catch`) → **async/await**.
- **Fetch API:** `fetch(url).then(res => res.json())`.

```
async function carregarDados() {
  try {
    const resposta = await fetch('https://api.exemplo.com/dados');
    const dados = await resposta.json();
    console.log(dados);
  } catch (erro) {
    console.error('Falha:', erro);
  }
}
```

Lesson 07 – Node.js e npm

- **Node.js:** runtime JavaScript no servidor (motor V8).
- **npm:** gestor de pacotes; `package.json`, `npm install`.
- **Módulos core:** `fs`, `http`, `path`, `os`.
- **CommonJS:** `require()` / `module.exports`.

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Olá, Mundo!\n');
});
server.listen(3000);
```

Lesson 08 – Express.js: Fundamentos

- **Express:** framework web minimalista.
- **Routing:** `app.get()` , `app.post()` , `app.put()` , `app.delete()` .
- **Objetos req / res :** `req.params` , `req.query` , `req.body` , `res.json()` , `res.status()` .
- **Middleware:** funções com acesso a `req` , `res` , `next` .

```
const express = require('express');
const app = express();
app.get('/users/:id', (req, res) => {
  res.send(`User ${req.params.id}`);
});
app.listen(3000);
```

Lesson 09 – Express.js: Middleware e Rotas Avançadas

- **Built-in:** `express.static()` , `express.json()` .
- **Terceiros:** `morgan` , `cors` , `helmet` .
- **Router:** modularização com `express.Router()` .
- **Middleware de erro:** assinatura `(err, req, res, next)` .

```
// users.js
const router = express.Router();
router.get('/', (req, res) => res.json([]));
module.exports = router;
// app.js
app.use('/users', usersRouter);
```

Lesson 10 – Bases de Dados: MongoDB e Mongoose

- **MongoDB:** NoSQL orientado a documentos (BSON).
- **Mongoose:** ODM – Schema, Model.
- **Schema:** define estrutura, validações (`required` , `min` , `unique`).
- **CRUD:** `Model.create()` , `find()` , `findByIdAndUpdate()` , `deleteOne()` .

```
const userSchema = new mongoose.Schema({
  nome: { type: String, required: true },
  email: { type: String, unique: true }
});
const User = mongoose.model('User', userSchema);
```

Lesson 11 – Autenticação e Sessões

- **Sessões:** `express-session` , cookie com session ID.
- **Bcrypt:** `hash()` e `compare()` para passwords.
- **Middleware de proteção:** verifica `req.session.userId` .

```
function requireAuth(req, res, next) {
  if (!req.session.userId) return res.status(401).json({error: 'Não autenticado'});
  next();
}
```

Lesson 12 – APIs RESTful

- **Princípios REST:** recursos (URIs), métodos HTTP, stateless.
- **Endpoints típicos:** `GET /users` , `POST /users` , `GET /users/:id` , `PATCH /users/:id` , `DELETE /users/:id` .
- **Códigos de estado:** 200, 201, 204, 400, 401, 404, 500.
- **JWT:** alternativa stateless para autenticação.

Lesson 13 – Segurança em Aplicações Web

- **OWASP Top 10:** injeção, XSS, CSRF, exposição de dados.
- **Mitigações:** validação/sanitização, escaping, helmet , cookies httpOnly / SameSite .
- **Rate limiting:** express-rate-limit .

⚠ **Informação crítica:** Nunca armazenar passwords em texto plano; usar sempre bcrypt. Validar todos os inputs no servidor.

Lesson 14 – Deployment e Boas Práticas

- **Variáveis de ambiente:** .env + dotenv .
- **Logging:** winston , pino .
- **PM2:** gestor de processos para produção.
- **HTTPS obrigatório.**
- **Opções de deploy:** VPS, Heroku, Railway, Docker.

📌 Lista de Termos-Chave

Termo	Definição resumida
HTTP	Protocolo de transferência de hipertexto; pedido-resposta.
HTML	Linguagem de marcação para estruturação de conteúdo.
CSS	Folhas de estilo para apresentação visual.
Flexbox	Layout unidimensional.
Grid	Layout bidimensional.
DOM	Representação em árvore do documento; API JS.
Evento	Sinal de interação (clique, tecla).
Assincronia	Promises, async/await.

Termo	Definição resumida
Node.js	JavaScript no servidor.
npm	Gestor de pacotes Node.
Express.js	Framework web para Node.
Middleware	Função que intercepta pedidos/respostas.
MongoDB	Base de dados NoSQL de documentos.
Mongoose	ODM para MongoDB.
Schema	Estrutura de documento no Mongoose.
Sessão	Mecanismo para manter estado (cookies).
Bcrypt	Hashing seguro de passwords.
API REST	Interface baseada em recursos e HTTP.
JWT	JSON Web Token – autenticação stateless.
OWASP	Projeto de segurança web.
XSS / CSRF	Ataques de injeção e falsificação de pedidos.
Helmet	Middleware de cabeçalhos de segurança.
PM2	Gestor de processos Node.js.
Deployment	Publicação da aplicação em produção.

Nota final: Este resumo baseia-se exclusivamente no conteúdo dos ficheiros de apontamentos do repositório `isel-leic-ipw/2425i-IPW-LEIC32D`. Possíveis ambiguidades estão assinaladas. Recomenda-se a consulta ao repositório original para exemplos completos e atualizações.